

Two products. One of them proved the other.

Waggle is an app for gig drivers, built for 92 countries. Populace is the testing tool we built to check it. Its first run found three defects in Waggle that a full manual test had missed, and two in Populace itself.

PRODUCT 01 · ANDROID, CONSUMER

Waggle

A free app for gig drivers. A Swiggy rider, an Uber driver and an Amazon Flex courier are often the same person, and no platform sees their whole day.

- **Earnings that follow the road.** Pay grows only with distance travelled, so a phone sitting in traffic earns nothing.
- **A map with people on it.** Friends' live positions and street search. Sharing stays off until the driver turns it on.
- **Works with no signal.** Posts and messages are saved on the phone and sent on reconnect.
- **Privacy the database enforces.** Other users can't read a driver's phone number, because the database refuses, not because the screen hides it.

43 languages, right-to-left included · 70 currencies · 92 countries · 33 gig platforms
React · TypeScript · Capacitor · PostgreSQL with row-level security on every table ·
8.2 MB download · Android 7.0+

In closed testing Free for drivers. No ads.

PRODUCT 02 · DEVELOPER TOOL

Populace

Simulated people use your app through its real API, all at once, so you can test what needs more than one person: feeds, live updates, and every permission rule you wrote.

- **One adapter file.** Thirteen small methods, two of them required. The engine knows how to be a person, and nothing about your product.
- **Front door only.** Real accounts through your API, with your permission rules applying. No admin keys, no direct database writes.
- **Honest coverage.** Coverage counts methods a run actually called. An empty method never counts.
- **Refuses production three ways.** The environment must declare itself, production hosts are refused however the URL is written, and an empty production list is flagged.

13-method contract · 0 runtime dependencies · 3 production checks · 121 self-tests, no backend needed
Node 18+ · median and 95th-percentile latency per method · JSON for CI plus a self-contained HTML report · exits non-zero on any problem

Proven — taking pilot users AGPL-3.0, source public.

Two runs, twelve days apart. The first shows what Populace finds. The second shows what a fixed app holds up to.

POPULACE REPORT · RUN 1 9 AUG 2026

✗ 5 problems found — run failed

5

DEFECTS FOUND

3m 30s

TIME TO FIND THEM

1

BLOCKED ALL SIGNUPS

Against a finished, signed app that had just passed a full manual test of every screen. Signup created no profile for any new user, because the write needed a column the app is deliberately not allowed to read. One like in four failed. Profile edits failed the same way. The other two defects were in Populace's own adapter.

What this shows: Populace finds real defects in software already considered done, names each cause in one line, and **exits non-zero** until they're fixed.

POPULACE REPORT · LARGEST CLEAN RUN 21 AUGUST 2026

✓ No failures across 932,455 API calls

0

FAILURES

932,455

API CALLS

13 / 13

METHODS COVERED

Two hundred simulated drivers in twenty cities across eleven countries, all on the backend at once for an hour. 2,271 km driven, 81,672 posts, 172,660 likes, 69,064 comments, 76,034 messages and 5,389 group joins. All 200 accounts created through real signup and deleted through real deletion, with nothing left behind.

What this shows: Waggle's permission rules hold up with many real accounts active at once, not just one person tapping through screens.

A later run put **300 drivers through 1,401,435 calls with zero API failures**. One call never left the test machine, so it's recorded as inconclusive, not clean. Three hundred is where throughput stops growing, **not where Waggle breaks**, which is still unfound. Both runs used a local backend, so they prove correctness, not network speed. Populace has also been run against Gitea, an app we didn't write.